MCI Lab 8

Ali Hussain Zaidi (sz10384)

Muhammad Haider Zaidi (mz10270)

Task # 1

Code:

```
#define GYRO_WHO_AM_I        0x0F    /* WHO_AM_I register */

/* Expected WHO_AM_I value */
#define I3G4250D_WHO_AM_I    0xD3    /* Expected device ID */

/* SPI command bits */
#define SPI_READ             0x80    /* MSB = 1 for read operation */

/* Chip Select (CS) pin control - PE3 */
#define CS_LOW()  HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET)
#define CS_HIGH() HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET)
```

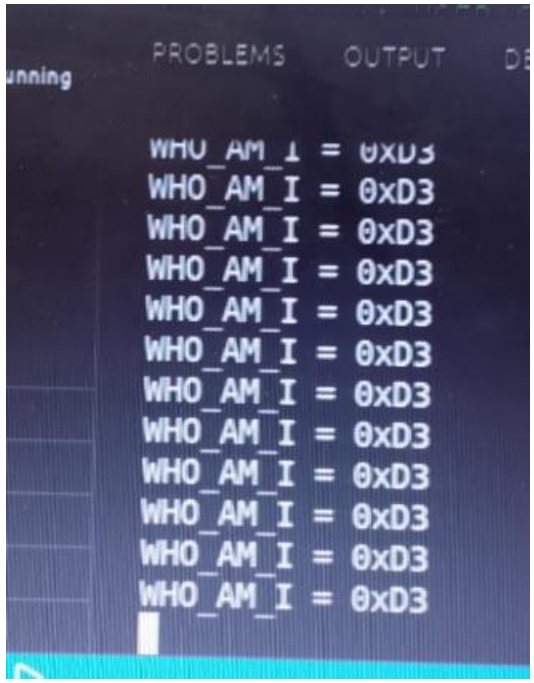
```
uint8_t SPI_ReadRegister(uint8_t reg)
{
    uint8_t tx_data[2];
    uint8_t rx_data[2] = {0};
    uint8_t result = 0;

    /* Prepare read command: register address with read bit (MSB=1) */
    tx_data[0] = SPI_READ | reg;  /* 0x80 | reg */
    tx_data[1] = 0x00;             /* Dummy byte to generate clock */

    /* Pull CS low to start communication */
    CS_LOW();

    /* Small delay for CS setup */
    for(volatile int i = 0; i < 10; i++);

    /* Send command and receive data */
    if(HAL_SPI_TransmitReceive(&hspi1, tx_data, rx_data, 2, HAL_MAX_DELAY) != HAL_OK)
    {
        CS_HIGH();
        return 0;
    }
```

```c
uint8_t SPI_ReadRegister(uint8_t reg)
{
    uint8_t tx_data[2];
    uint8_t rx_data[2] = {0};
    uint8_t result = 0;

    /* Prepare read command: register address with read bit (MSB=1) */
    tx_data[0] = SPI_READ | reg;  /* 0x80 | reg */
    tx_data[1] = 0x00;            /* Dummy byte to generate clock */

    /* Pull CS low to start communication */
    CS_LOW();

    /* Small delay for CS setup */
    for(volatile int i = 0; i < 10; i++);

    /* Send command and receive data */
    if(HAL_SPI_TransmitReceive(&hspi1, tx_data, rx_data, 2, HAL_MAX_DELAY) != HAL_OK)
    {
        CS_HIGH();
        return 0;
    }

    /* Pull CS high to end communication */
    CS_HIGH();

    /* Second byte contains the register value */
    result = rx_data[1];

    return result;
}
```

```c
void cout(const char* fmt, ...)
{
    char buffer[256];
    va_list args;
    va_start(args, fmt);
    int len = vsnprintf(buffer, sizeof(buffer), fmt, args);
    va_end(args);

    if (len > 0)
    {
        HAL_UART_Transmit(&huart2, (uint8_t*)buffer, len, HAL_MAX_DELAY);
    }
}
```

```c
int main(void)
{

    HAL_Init();

    SystemClock_Config();

    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_SPI1_Init();  /* Use the default initialization first */
    MX_USART2_UART_Init();
    MX_USB_PCD_Init();


    uint8_t who_am_i;

    HAL_Delay(100);

    /* Print header */
    cout("\r\n========================================\r\n");
    cout("Lab 8 Task 1: SPI Communication Test\r\n");
    cout("========================================\r\n");

    /* Initialize the gyroscope and read WHO_AM_I */
    who_am_i = SPI_ReadRegister(GYRO_WHO_AM_I);

    /* Display the result */
    cout("WHO_AM_I Register (0x%02X) = 0x%02X\r\n", GYRO_WHO_AM_I, who_am_i);

    /* Verify the value */
    if(who_am_i == I3G4250D_WHO_AM_I)
    {
        cout("SUCCESS: Detected I3G4250D Gyroscope\r\n");
    }
    else if(who_am_i == 0xD4)
    {
        cout("SUCCESS: Detected L3GD20 Gyroscope (Green Board)\r\n");
    }
    else
    {
        cout("ERROR: Unexpected WHO_AM_I value! Expected 0x%02X\r\n", I3G4250D_WHO_AM_I);
        cout("Check SPI connections and configuration.\r\n");
    }

    cout("========================================\r\n");

    while (1)
    {
        /* Periodically read and display WHO_AM_I to verify communication */
        uint8_t current_id = SPI_ReadRegister(GYRO_WHO_AM_I);
        cout("WHO_AM_I = 0x%02X\r\n", current_id);

        /* Read every 1 second */
        HAL_Delay(1000);

        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}
```

Output:



Task # 2

Code:

```
#define CTRL_REG1           0x20    /* Control register 1 */
#define CTRL_REG1_VAL       0x0F    /* PD=1 (power on), Zen=1, Yen=1, Xen=1 */

#define OUT_TEMP_ADDR       0x26    /* Temperature output register */

/* SPI Command Bits */
#define SPI_READ_FLAG       0x80    /* Bit7 = 1 => read operation */

/* Chip Select Pin - PE3 on STM32F3 Discovery */
#define GYRO_CS_PIN         GPIO_PIN_3
#define GYRO_CS_PORT        GPIOE
```

```
void gyro_init(void)
{
    uint8_t tx[2];
    tx[0] = CTRL_REG1;       /* Register address to write to */
    tx[1] = CTRL_REG1_VAL;   /* Value: PD=1, Zen=1, Yen=1, Xen=1 */

    GYRO_CS_LOW();
    HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);
    GYRO_CS_HIGH();
}
```

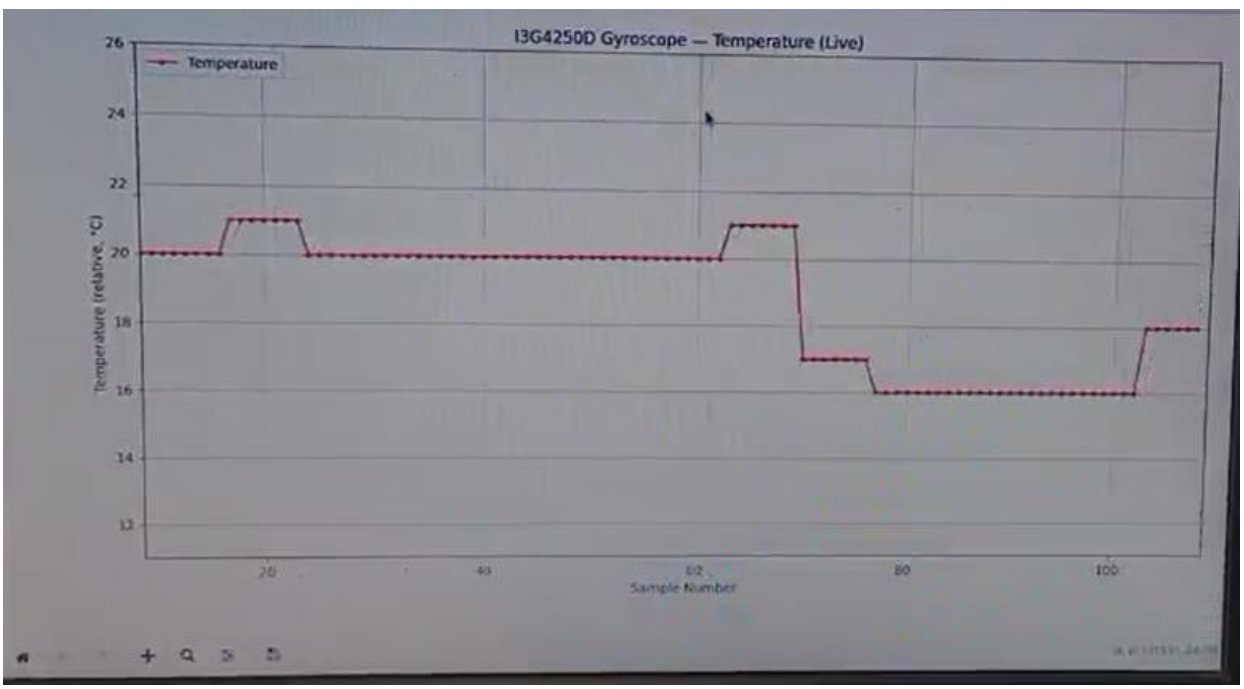
```c
void gyro_read_temp_start(void)
{
    tx_buf[0] = SPI_READ_FLAG | OUT_TEMP_ADDR;  /* 0xA6 */
    spiState  = SPI_STATE_IDLE;

    GYRO_CS_LOW();
    HAL_SPI_Transmit_IT(&hspi1, tx_buf, 1);     /* Non-blocking TX */
}
```

```c
typedef enum {
    SPI_STATE_IDLE = 0,
    SPI_STATE_TX_DONE,
    SPI_STATE_RX_DONE
} SPI_State_t;

volatile SPI_State_t spiState = SPI_STATE_IDLE;

/* SPI Buffers */
uint8_t tx_buf[1];
uint8_t rx_buf[1];

/* Temperature value (signed 8-bit per datasheet) */
volatile int8_t temperature = 0;

/* UART transmit buffer */
char uart_buf[32];
```

```c
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi)
{
    if (hspi->Instance == SPI1)
    {
        spiState = SPI_STATE_TX_DONE;
        HAL_SPI_Receive_IT(&hspi1, rx_buf, 1);  /* Non-blocking RX */
    }
}

/**
  * @brief  SPI Receive Complete Callback (auto-called by HAL after RX done).
  *         The temperature byte is now in rx_buf[0].
  *         Pull CS HIGH to end the SPI transaction, then store the value.
  */
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi)
{
    if (hspi->Instance == SPI1)
    {
        GYRO_CS_HIGH();

        temperature = (int8_t)rx_buf[0];   /* Cast to signed (datasheet: -1 °C/digit) */
        spiState    = SPI_STATE_RX_DONE;
    }
}
```

```c
while (1)
{
    /* --- Step 1: Kick off interrupt-based temperature read --- */
    gyro_read_temp_start();

    /* --- Step 2: Wait (spin) until the full SPI transaction completes ---
     *   The HAL callbacks advance spiState:
     *     IDLE -> TX_DONE (in TxCpltCallback)
     *          -> RX_DONE (in RxCpltCallback)
     */
    while (spiState != SPI_STATE_RX_DONE)
    {
        /* Waiting for interrupt callbacks to finish */
    }

    /* --- Step 3: Transmit temperature value over UART ---
     *   Format: signed integer followed by \r\n
     *   Example output line: "27\r\n"
     *   The Python plotting script on the LMS parses this format.
     */
    int len = snprintf(uart_buf, sizeof(uart_buf), "%d\r\n", (int)temperature);
    HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf, (uint16_t)len, HAL_MAX_DELAY);

    /* --- Step 4: Wait ~150 ms before next reading (100-200 ms per lab spec) --- */
    HAL_Delay(150);

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Output:



Task #3

Code:

```
/* USER CODE BEGIN PD */
#define CTRL_REG1      0x20
#define CTRL_REG1_VAL  0x8F   /* PD=1, Zen=1, Yen=1, Xen=1, ODR=400Hz */

#define CTRL_REG4      0x23
#define CTRL_REG4_VAL  0x00   /* FS = ±245 dps */

#define OUT_TEMP       0x26
#define OUT_X_L        0x28

#define SPI_READ       0x80
#define SPI_AUTO_INC   0x40

#define GYRO_CS_PIN    GPIO_PIN_3
#define GYRO_CS_PORT   GPIOE
/* USER CODE END PD */

/* USER CODE BEGIN PM */
#define GYRO_CS_LOW()   HAL_GPIO_WritePin(GYRO_CS_PORT, GYRO_CS_PIN, GPIO_PIN_RESET)
#define GYRO_CS_HIGH()  HAL_GPIO_WritePin(GYRO_CS_PORT, GYRO_CS_PIN, GPIO_PIN_SET)
```

```
static void gyro_write_reg(uint8_t reg, uint8_t val)
{
    uint8_t tx[2] = { reg & 0x3F, val };
    GYRO_CS_LOW();
    HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);
    GYRO_CS_HIGH();
}

/* Read one register */
static uint8_t gyro_read_reg(uint8_t reg)
{
    uint8_t cmd = SPI_READ | (reg & 0x3F);
    uint8_t data = 0;
    GYRO_CS_LOW();
    HAL_SPI_Transmit(&hspi1, &cmd,  1, HAL_MAX_DELAY);
    HAL_SPI_Receive (&hspi1, &data, 1, HAL_MAX_DELAY);
    GYRO_CS_HIGH();
    return data;
}
```

```c
static int write_dps(char *buf, int32_t raw)
{
    /* Scale to 0.01 dps units: raw * 875 / 100  (= raw * 8.75) */
    int32_t scaled = (raw * 875L) / 100L;   /* units of 0.01 dps */

    int negative = (scaled < 0);
    if (negative) scaled = -scaled;

    int32_t whole = scaled / 100;           /* integer dps part   */
    int32_t frac  = scaled % 100;           /* two decimal places */

    if (negative)
        return sprintf(buf, "-%ld.%02ld", (long)whole, (long)frac);
    else
        return sprintf(buf,  "%ld.%02ld", (long)whole, (long)frac);
}
```

```c
    while (1)
    {
        gyro_read_all();

        /* Build output string using only integer sprintf (%ld, %d) */
        char xs[12], ys[12], zs[12];
        write_dps(xs, x_raw);
        write_dps(ys, y_raw);
        write_dps(zs, z_raw);

        int len = sprintf(uart_buf, "%d, %s, %s, %s\r\n",
                          (int)temp_raw, xs, ys, zs);

        HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf, (uint16_t)len, HAL_MAX_DELAY);

        HAL_Delay(100);
        /* USER CODE END WHILE */
        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}
```

```c
static void MX_SPI1_Init(void)
{
    hspi1.Instance               = SPI1;
    hspi1.Init.Mode              = SPI_MODE_MASTER;
    hspi1.Init.Direction         = SPI_DIRECTION_2LINES;
    hspi1.Init.DataSize          = SPI_DATASIZE_8BIT;
    hspi1.Init.CLKPolarity       = SPI_POLARITY_LOW;
    hspi1.Init.CLKPhase          = SPI_PHASE_1EDGE;
    hspi1.Init.NSS               = SPI_NSS_SOFT;
    hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_4;
    hspi1.Init.FirstBit          = SPI_FIRSTBIT_MSB;
    hspi1.Init.TIMode            = SPI_TIMODE_DISABLE;
    hspi1.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE;
    hspi1.Init.CRCPolynomial     = 7;
    hspi1.Init.CRCLength         = SPI_CRC_LENGTH_DATASIZE;
    hspi1.Init.NSSPMode          = SPI_NSS_PULSE_ENABLE;
    if (HAL_SPI_Init(&hspi1) != HAL_OK) Error_Handler();
}
```

Output:

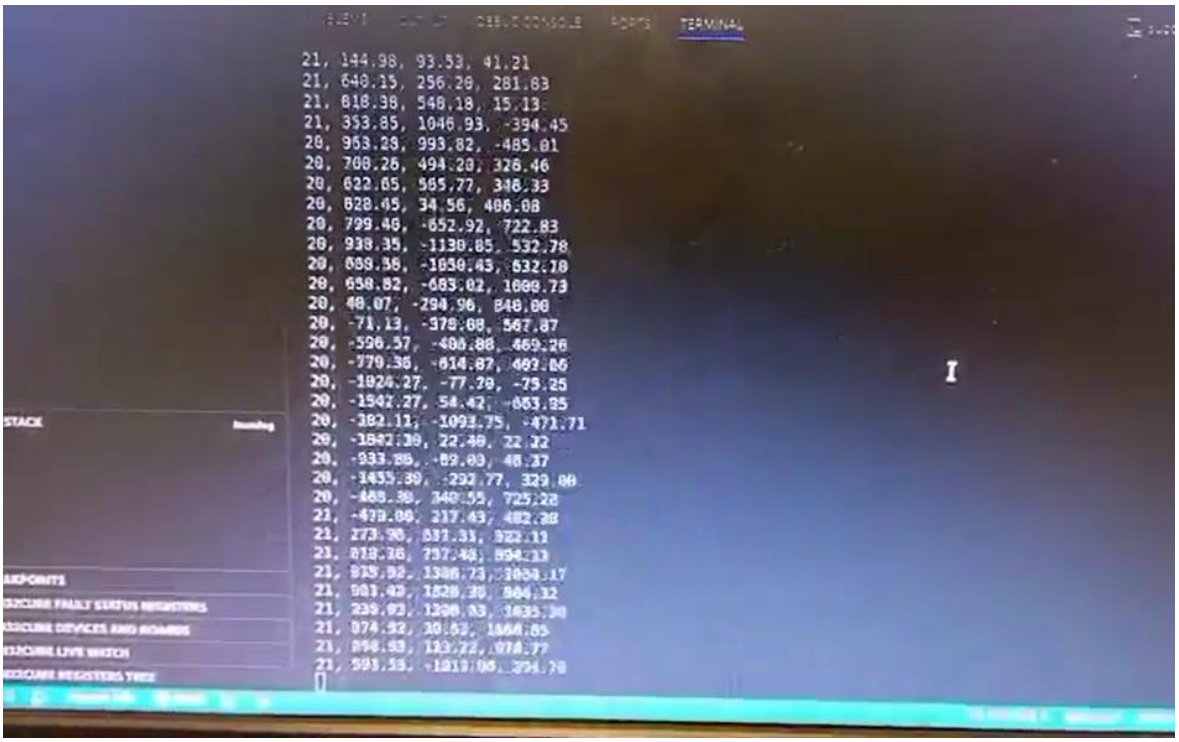

TERMINAL
21, 144.98, 93.53, 41.21
21, 640.15, 256.20, 281.83
21, 818.38, 548.18, 15.13
21, 353.85, 1046.93, -394.45
20, 953.28, 993.82, -485.01
20, 708.26, 494.20, 326.46
20, 622.85, 565.77, 348.33
20, 828.45, 34.56, 406.08
20, 799.46, -652.92, 722.83
20, 938.35, -1130.85, 532.78
20, 888.38, -1050.43, 632.10
20, 650.82, -683.02, 1000.73
20, 40.07, -294.96, 840.00
20, -71.13, -378.08, 567.87
20, -596.57, -400.88, 469.26
20, -770.36, -614.87, 409.86
20, -1024.27, -77.70, -75.25
20, -1947.27, 54.42, -663.85
20, -382.11, -1093.75, -471.71
20, -1842.30, 22.40, 22.22
20, -933.86, -89.80, 40.37
20, -1455.30, -292.77, 329.00
20, -488.30, 340.55, 725.28
21, -479.00, 217.43, 482.28
21, 273.98, 831.31, 922.11
21, 818.36, 797.43, 894.13
21, 875.92, 1306.73, 1054.17
21, 901.43, 1528.30, 984.32
21, 239.02, 1290.03, 1035.30
21, 874.52, 30.52, 1066.85
21, 894.93, 113.22, 978.77
21, 593.53, -1013.00, 394.70

Figure 1
I3G4250D Gyroscope — Live Data
Temperature (Live)
temperature
Temperature (relative, degree C)
22
20
18
16
14
180
200
220
240
260
Sample Number
Angular Velocity (Live)
X (dps)
Y (dps)
Z (dps)
Angular Velocity (dps)
3000
2000
1000
0
-1000
-2000
180
200
220
240
260
Sample Number

Q) You read a WHO AM I value of 0x00 from the gyroscope sensor instead of the expected ID. List two possible causes for this issue.

If the SPI polarity and phase do not match the sensor's requirements, the data lines may not be sampled correctly, resulting in all-zero reads.

A poor solder joint, loose wire, or missing pull-up resistor on the MISO line can leave the pin floating, causing the microcontroller to read a constant logic low (0x00) when the sensor tries to transmit data.